

Notes de révision — Évaluation L12

Cours : Systèmes d'exploitation de type Unix **Évaluation** : Leçon 12 — Shell + Virtualisation **Date** : 2026-05-06 (5 périodes) **Document** : Fiche de révision dense (toutes les commandes, syntaxes et pièges)

Sommaire

Partie 1 — Shell

1. Commandes de base & navigation
2. Globbing & expansions
3. Redirections, pipes & chaînage
4. Filtres de texte (grep, cut, awk, ...)
5. Recherche avec `find`
6. Permissions & droits d'accès
7. Processus & jobs
8. Variables & environnement
9. Scripts Bash
10. Here-documents
11. Alias
12. Gestion utilisateurs & groupes

Partie 2 — Virtualisation

1. Concepts (KVM, libvirt, conteneurs vs VMs)
2. `virsh` — gestion des VMs
3. `virt-install` — création de VMs
4. Stockage (raw, qcow2, LVM)
5. Clonage de VMs
6. LVM — extension, snapshots
7. Réseaux virtuels libvirt
8. Configuration réseau dans la VM
9. SSH (jump host, tunnel, clés)
10. Post-installation d'une VM
11. Incus (LXD fork)

Annexes

- Cheatsheet 1 page
 - Pièges fréquents au test
-

PARTIE 1 — SHELL

1. Commandes de base & navigation

Identité & système

Commande	Effet
<code>whoami</code>	nom de l'utilisateur courant
<code>id</code> / <code>id user</code>	UID + GID + groupes secondaires
<code>who</code> / <code>w</code> / <code>users</code>	utilisateurs connectés
<code>tty</code>	terminal courant
<code>hostname</code>	nom de la machine
<code>date</code>	date courante
<code>pwd</code>	répertoire de travail
<code>echo \$LOGNAME</code> / <code>echo \$TERM</code>	variables d'environnement
<code>printenv</code>	toutes les variables d'env
<code>df -h</code>	espace disque
<code>clear</code>	nettoyer le terminal

Listage & navigation

```
ls          # entrées visibles
ls -a       # inclure cachés (.fichier)
ls -l       # format long (perm, propriétaire, taille, date)
ls -la      # combiné
ls -l       # une entrée par ligne
ls -lh      # tailles humaines (K, M, G)

cd          # = cd ~ (home)
cd ~user    # home d'un autre utilisateur
cd -        # répertoire précédent
cd ..       # dossier parent
```

Création / copie / déplacement / suppression

```
mkdir test                # créer un dossier
mkdir -p a/b/c            # créer toute l'arborescence
touch fichier.txt         # créer un fichier vide ou maj date

cp source dest            # copier un fichier
cp -r dossier1 dossier2   # copier un dossier (récursif)
mv ancien nouveau        # renommer / déplacer
rm fichier               # supprimer un fichier
rm -r dossier             # supprimer récursivement
rm -rf dossier            # forcer + récursif (DANGER)

ln cible lien             # lien physique (hard link)
ln -s cible lien          # lien symbolique (soft link)

file fichier             # détecter le type d'un fichier
file -L lien              # suivre les liens symboliques
```

2. Globbing & expansions

Caractères génériques (wildcards)

Motif	Signification
<code>*</code>	n'importe quelle chaîne (y compris vide)
<code>?</code>	exactement 1 caractère
<code>[abc]</code>	un caractère parmi a, b, c
<code>[a-z]</code>	un caractère dans la plage
<code>[!abc]</code> ou <code>[^abc]</code>	un caractère sauf a, b, c

Expansion d'accolades

```
touch a{1..10}           # crée a1, a2, ..., a10
mkdir trav{1..3}         # crée trav1, trav2, trav3
cp /usr/include/{a,c,m}*.h ~/trav1 # liste explicite
```

Piège — Le shell expand AVANT d'invoquer la commande

```
find /usr/include -name al*.h    # DANGEREUX : si al*.h existe localement, il est sub-
stitu  
find /usr/include -name 'al*.h'  # CORRECT : guillemets → motif pass   tel quel    find
```

R  gle d'or : avec `find`, **toujours** entourer le motif de guillemets simples.

3. Redirections, pipes & chaînage

Descripteurs

FD	Nom	Symbole
0	stdin	<
1	stdout	> (écrase), >> (append)
2	stderr	2> , 2>>

Exemples

```
ls > listing.txt           # stdout dans fichier (écrase)
cat /etc/passwd >> out.txt  # ajout à la fin
cut -d: -f1 < /etc/passwd  # stdin depuis fichier
find / 2>/dev/null         # supprimer les erreurs
ls /no /tmp > out 2>&1      # stdout + stderr → out
ls /no /tmp &> out          # raccourci bash
```

Pipe |

```
cat /etc/passwd | cut -d: -f1 | sort | uniq
ls | wc -l
ps -ef | grep ssh
```

tee — duplique stdout

```
ls -l | tee fichier.log | grep ".txt"
# affiche, écrit dans fichier.log, et continue
```

Chaînage logique

Op.	Effet
;	exécute toujours la suivante
&&	exécute la suivante seulement si succès (<code>\$? == 0</code>)
	exécute la suivante seulement si échec (<code>\$? != 0</code>)

```
make && ./run              # run uniquement si make réussit
ping -c1 host || echo "K0"
```

Code de sortie

```
cmd; echo $?              # 0 = succès, ≠0 = erreur
exit 0                    # dans un script, sortir avec un code
```

4. Filtres de texte

cat, head, tail, less

```
cat fichier          # affiche tout
head fichier         # 10 premières lignes
head -n 5 fichier    # 5 premières
tail -n 20 fichier   # 20 dernières
tail -f log          # suivre en temps réel
less fichier         # pagination interactive
```

grep — recherche par motif

```
grep motif fichier   # lignes contenant "motif"
grep -i motif        # ignorer la casse
grep -v motif        # inverser (lignes SANS le motif)
grep -n motif        # afficher numéros de ligne
grep -r motif dossier/ # récursif
grep -E "regex"      # regex étendue
grep "^li" /etc/passwd # commence par li
grep ":3$" /etc/passwd # finit par :3
grep "1[0-9][0-9]$"  # 3 chiffres commençant par 1
```

cut — extraire des champs

```
cut -d: -f1 /etc/passwd # 1er champ, séparateur :
cut -d: -f1,6 /etc/passwd # champs 1 et 6
cut -c1-10 fichier      # caractères 1 à 10
```

tr — translation/suppression

```
echo "abc" | tr 'a-z' 'A-Z' # → ABC
tr -s ' '                  # squeeze : espaces multiples → 1
tr -d ' '                  # supprimer espaces
```

sort, uniq, wc

```
sort fichier         # tri alphabétique
sort -n              # numérique
sort -r              # décroissant
sort -k2 -t,         # 2e colonne, séparateur virgule
sort fichier | uniq   # supprime doublons consécutifs
sort fichier | uniq -c # avec compteur
wc -l fichier         # nombre de lignes
wc -w / wc -c         # mots / caractères
```

awk — traitement par colonnes

```
awk '{print $1}' fichier # 1re colonne
awk -F: '{print $1, $3}' /etc/passwd # séparateur :
awk '$3 > 1000' /etc/passwd # filtrer
awk '{sum += $1} END {print sum}' # somme
w | awk '{print $1, $8}' # user + commande
```

diff, cmp

```
diff a.txt b.txt          # différences ligne par ligne
diff -y a.txt b.txt      # côte à côte
cmp a.bin b.bin          # comparaison binaire
```

5. Recherche avec find

```
find . -name "*.txt"      # par nom (guillemets!)
find . -iname "*.TXT"    # insensible à la casse
find . -type f           # fichiers ordinaires
find . -type d           # dossiers
find . -type l           # liens symboliques
find . -size +1M         # > 1 MiB
find . -mtime -7         # modifié dans les 7 derniers jours
find . -mtime +30        # > 30 jours
find . -user lucien      # par propriétaire
find . -group users      # par groupe
find . -perm 644         # permissions exactes
find . -perm -111        # AU MOINS x pour tous (-)
find . -perm -u=x,g=x,o=x # idem en littéral

# Actions
find . -name "*.tmp" -delete
find . -name "*.sh" -exec chmod +x {} \;
find . -type f -exec grep -l motif {} \;
```

Pour **-perm** : préfixe **-** = "au moins ces droits", pas de préfixe = "exactement ces droits",
/ = "au moins un de ces droits".

6. Permissions & droits d'accès

Lecture d'un ls -l

```
-rw-r--r-- 1 lucien users 1234 Mar 18 12:00 fichier.txt
|  |  |  |
|  u  g  o
type
```

Type	Sens
-	fichier ordinaire
d	répertoire
l	lien symbolique
b, c	périphérique bloc/caractère

Conversion octal ↔ littéral

Oct	Bin	rwX
0	000	---
1	001	--X
2	010	-W-
3	011	-WX
4	100	r--
5	101	r-X
6	110	rw-
7	111	rwX

chmod

```
# Forme littérale : cibles (u/g/o/a) + opérateur (+/-/=) + droits (r/w/x)
chmod u+x script.sh
chmod g-w fichier
chmod a=r fichier
chmod u=rwx,g=rx,o= dossier

# Forme octale
chmod 644 fichier      # rw-r--r--
chmod 755 dossier      # rwxr-xr-x
chmod 600 ~/.ssh/id_rsa # rw-----
chmod 700 ~            # rwx-----
```

chown , chgrp

```
sudo chown lucien fichier
sudo chown lucien:users fichier      # propriétaire ET groupe
sudo chown -R lucien dossier/       # récursif
sudo chgrp users fichier
```

umask — masque par défaut

- Droits de base : **666** pour fichiers, **777** pour dossiers
- Droits effectifs = base – umask

```
umask          # afficher (ex: 0022)
umask 0077     # nouveau masque (très restrictif)

# Avec umask 0022 : fichier 644, dossier 755
# Avec umask 0077 : fichier 600, dossier 700
```

Rôle critique du **x** sur un répertoire

Droits dossier	Possibilités
drwx	tout : lister + traverser + créer/supprimer
dr-x	lister + traverser + lire fichiers connus, mais pas créer
d-wx	créer/supprimer + traverser, mais pas lister
d--x	traverser et accéder aux fichiers connus uniquement
dr--	voir les noms mais pas entrer (inutilisable)
d-w-	INUTILE (aucun accès sans x)

w sans **x** sur un répertoire ne sert à rien.

Droits minimums pour des opérations

Opération	Droits dossier	Droits fichier
cat src/foo.txt	--x sur src	r-- sur foo
cp src/foo dst/	--x sur src, -wx sur dst	r-- sur foo
mv src/foo dst/	-wx sur src, -wx sur dst	r-- sur foo
ls dossier/	r--	—
cd dossier/	--x	—

7. Processus & jobs

```
ps                # processus du terminal courant
ps -ef            # tous les processus, format complet
ps -aux           # tous + ressources
ps -axf           # arborescence parent/enfant
top               # en temps réel (q pour quitter)
htop              # version améliorée

kill PID          # SIGTERM (15) – arrêt propre
kill -9 PID       # SIGKILL – forcé
kill -l           # liste des signaux
killall nom       # tuer tous les proc de ce nom
pgrep nom         # trouver les PID
```

Jobs (arrière-plan)

```
cmd &             # lancer en arrière-plan
Ctrl+Z            # suspendre (SIGTSTP)
Ctrl+C            # interrompre (SIGINT)
jobs              # lister les jobs
fg %1             # ramener job 1 au premier plan
bg %1             # reprendre job 1 en arrière-plan
kill %1           # tuer job 1
nohup cmd &       # survivre à la déconnexion
disown            # détacher un job du shell
```


Variables liées aux processus

- `$$` : PID du shell courant
- `$!` : PID du dernier processus lancé en arrière-plan
- `$?` : code de sortie de la dernière commande

8. Variables & environnement

Variables shell vs variables d'environnement

```
var="valeur"          # locale au shell
export var             # exportée aux sous-processus
export var="valeur"    # combiné
unset var              # supprimer

echo $var              # référencer
echo "${var}suffixe"   # accolades pour délimiter
```

Variables système courantes

Variable	Sens
<code>\$HOME</code>	dossier personnel
<code>\$USER</code> / <code>\$LOGNAME</code>	utilisateur
<code>\$UID</code>	UID numérique
<code>\$PATH</code>	chemins de recherche
<code>\$PWD</code>	répertoire courant
<code>\$SHELL</code>	shell de login
<code>\$PS1</code>	invite primaire
<code>\$IFS</code>	séparateurs de champs (défaut espace/tab/newline)

Persistance dans `~/.bashrc` ou `~/.profile`

```
echo 'export PATH=$PATH:~/bin' >> ~/.bashrc
echo "alias ll='ls -la'" >> ~/.bashrc
source ~/.bashrc          # recharger sans rouvrir le terminal
. ~/.bashrc               # même chose, syntaxe POSIX
```

Sourcer vs exécuter un script

- **Exécuter** (`./script.sh`) : nouveau shell → variables disparaissent à la fin
- **Sourcer** (`source script.sh` ou `. script.sh`) : exécuté dans le shell courant → variables persistent

9. Scripts Bash

Squelette

```
#!/bin/bash
# commentaire

set -e          # arrêt immédiat sur erreur (recommandé)
set -u          # erreur si variable indéfinie
set -x          # mode debug (affiche chaque commande)

echo "Hello $USER"
exit 0
```

Rendre exécutable : `chmod u+x script.sh`, puis `./script.sh`.

Paramètres positionnels

Var	Sens
<code>\$0</code>	nom du script
<code>\$1</code> , <code>\$2</code> , ...	arguments 1, 2, ...
<code>\$@</code>	tous les arguments (chacun quoté)
<code>\$*</code>	tous les arguments (1 chaîne)
<code>\$#</code>	nombre d'arguments
<code>shift</code>	décale <code>\$1 ← \$2 ← \$3 ...</code>

Tests

```
# Numérique : -eq -ne -lt -le -gt -ge
[ "$n" -eq 5 ]

# Chaîne : = != -z (vide) -n (non vide)
[ "$s" = "abc" ]
[ -z "$s" ]

# Fichier : -e (existe) -f (régulier) -d (dossier) -r -w -x -L (lien)
[ -f "$f" ]

# Bash étendu (préférable)
[[ $s == *7* ]]          # contient "7"
[[ $s =~ ^[0-9]+$ ]]     # regex
(( n > 5 ))               # arithmétique
```

Structures de contrôle

```
# IF
if [ "$x" -gt 0 ]; then
    echo "positif"
elif [ "$x" -eq 0 ]; then
    echo "zéro"
else
    echo "négatif"
fi

# FOR
for i in 1 2 3; do echo $i; done
for i in {1..10}; do echo $i; done
for i in $(seq 1 10); do echo $i; done
for f in *.txt; do echo "$f"; done

# WHILE
i=1
while [ $i -le 100 ]; do
    echo $i
    i=$((i + 1))
done

# UNTIL (inverse de while)
until [ $i -gt 10 ]; do echo $i; i=$((i+1)); done

# CASE
case "$1" in
    start) echo "démarrage" ;;
    stop)  echo "arrêt" ;;
    *)     echo "usage: $0 {start|stop}" ;;
esac
```

Arithmétique

```
$((3 + 2))          # 5
$((10 / 3))         # 3 (entier)
$((10 % 3))         # 1
i=$((i + 1))
((i++))             # incrémenter
```

Fonctions

```
ma_fonction() {
    local var="local"
    echo "arg1=$1, arg2=$2"
    return 0          # code de retour
}

ma_fonction toto titi
echo $?              # récupère le return
```

Lecture interactive

```
read -p "Nom : " nom
read -s -p "Password : " pw    # -s = silencieux
read a b c                    # 3 variables d'un coup
```

Exemple — table de multiplication

```
#!/bin/bash
n=$1
for i in $(seq 1 12); do
    printf "%2d x %2d = %3d\n" $i $n $((i * n))
done
```

Exemple — création de 100 utilisateurs

```
#!/bin/bash
ACTION=$1
i=1
while [ $i -le 100 ]; do
    USER="user$i"
    case "$ACTION" in
        create) useradd -m "$USER" ;;
        delete) userdel -r "$USER" ;;
        *) echo "usage: $0 {create|delete}"; exit 1 ;;
    esac
    i=$((i + 1))
done
```

Validation : **shellcheck**

```
shellcheck script.sh          # détecte erreurs et mauvaises pratiques
```

10. Here-documents

```
cat <<EOF
Bonjour $USER
EOF
# → "Bonjour lucien" (variables interpolées)

cat <<'EOF'
Bonjour $USER
EOF
# → "Bonjour $USER" (littéral, à cause des quotes)

cat <<-EOF
    Texte avec
    tabulations supprimées en début
EOF
# (le tiret fait sauter les tabulations seulement)

# Rediriger vers un fichier
cat <<EOF > config.conf
host=localhost
port=8080
EOF

# Capturer dans une variable
msg=$(cat <<EOF
ligne 1
ligne 2
EOF
)
```

11. Alias

```
alias ll='ls -la'
alias ..='cd ..'
alias fin='tail -2'

alias                # lister tous
alias ll             # afficher la définition
unalias ll           # supprimer

# Persistance
echo "alias ll='ls -la'" >> ~/.bashrc
source ~/.bashrc
```

12. Gestion utilisateurs & groupes

Fichiers système

Fichier	Contenu
/etc/passwd	user:x:UID:GID:GECOS:home:shell
/etc/shadow	mots de passe hashés (root only)
/etc/group	groupe:x:GID:membres
/etc/gshadow	mots de passe de groupe

Création / suppression / modification

```
sudo adduser lucien          # interactif (Debian/Ubuntu)
sudo useradd -m lucien       # non interactif, crée home
sudo useradd -m -s /bin/bash -G users,sudo lucien
sudo userdel -r lucien       # supprime + home

sudo passwd lucien           # définir/changer mot de passe
sudo chage -l lucien         # info expiration

sudo groupadd stock
sudo groupdel stock

sudo usermod -aG stock lucien # AJOUTER au groupe (-a obligatoire avec -G)
sudo usermod -g users lucien  # changer groupe PRIMAIRE
sudo usermod -L lucien        # verrouiller
sudo usermod -U lucien        # déverrouiller
```

Piège : `usermod -G groupe user` **remplace** tous les groupes secondaires. Toujours utiliser `-aG`.

Inspection

```
id lucien                # UID, GID, groupes
groups lucien             # juste les groupes
getent passwd lucien      # entrée /etc/passwd
who / w                   # connectés
```

Changement de groupe d'un fichier

<code>chgrp users fichier</code>	<code># OK si membre de "users"</code>
<code>chmod g=rw fichier</code>	<code># ajuster les droits du groupe</code>

PARTIE 2 — VIRTUALISATION

1. Concepts fondamentaux

Notion	Description
Virtualisation	Abstraction du matériel pour exécuter plusieurs systèmes sur un même hôte
Hyperviseur type 1	"Bare metal", direct sur le matériel (KVM, ESXi, Hyper-V)
Hyperviseur type 2	Sur un OS hôte (VirtualBox, VMware Workstation)
KVM	Module noyau Linux qui transforme le kernel en hyperviseur de type 1
QEMU	Émulateur de matériel (disques, réseau, USB), couplé à KVM
libvirt	Couche d'abstraction qui pilote KVM/QEMU/Xen/LXC...
virsh	CLI de libvirt
virt-manager	GUI de libvirt
virt-install	Outil de création/install de VMs

VMs vs conteneurs

Aspect	VM (KVM)	Conteneur (Incus/LXD)
Kernel	Indépendant	Partagé avec l'hôte
Isolation	Forte (CPU virtualisé)	Légère (cgroups, namespaces)
Démarrage	~secondes/dizaines de s	~ms
Empreinte	RAM/disque importants	Légère
Usage	OS hétérogènes, isolation forte	Microservices, dev, déploiement rapide

URI de connexion libvirt

```
qemu:///system          # local, mode système
qemu:///session         # local, sans privilèges
qemu+ssh://user@host/system  # distant via SSH
```

2. **virsh** — gestion des VMs

Lister & informations

```
virsh list              # VMs actives
virsh list --all        # toutes (y compris arrêtées)
virsh list --all --name # juste les noms
virsh dominfo vm1       # infos générales
virsh domifaddr vm1     # adresses IP
virsh domblklist vm1    # disques attachés
virsh domiflist vm1     # interfaces réseau
virsh dumpxml vm1       # config XML complète
virsh dumpxml vm1 > vm1.xml # sauvegarder
```

Cycle de vie

```
virsh start vm1           # démarrer
virsh shutdown vm1       # arrêt propre (ACPI)
virsh destroy vm1        # arrêt forcé (équivalent unplug)
virsh reboot vm1         # redémarrer
virsh suspend vm1        # suspendre (RAM)
virsh resume vm1         # reprendre
virsh autostart vm1       # démarrer au boot de l'hôte
virsh autostart --disable vm1
```

Définition & suppression

```
virsh define vm1.xml      # enregistrer depuis XML
virsh undefine vm1        # supprimer la définition (pas les disques)
virsh undefine vm1 --remove-all-storage # + supprimer les disques
virsh edit vm1            # éditer le XML (dans $EDITOR)
```

Console

```
virsh console vm1         # console série (Ctrl+] pour quitter)
virsh vncdisplay vm1      # port VNC s'il existe
```

3. virt-install — création de VMs

Squelette

```
virt-install --connect qemu:///system \
  --name vm1 \
  --memory 1024 \
  --vcpus 1 \
  --disk path=/var/lib/libvirt/images/vm1.img,format=raw,bus=virtio,size=6 \
  --location ~/iso/debian-13.3.0-amd64-netinst.iso \
  --network network=default,model=virtio \
  --osinfo name=debian11 \
  --graphics none \
  --console pty,target_type=serial \
  --extra-args "console=ttyS0,115200n8"
```


Options clés

Option	Effet
<code>--name</code>	nom logique de la VM
<code>--memory MB</code>	RAM en mégaoctets
<code>--vcpus N</code>	nombre de CPU virtuels
<code>--disk path=...,size=...,format=...,bus=virtio</code>	disque principal
<code>--cdrom ISO</code>	ISO d'installation (graphique)
<code>--location ISO_OU_URL</code>	ISO ou mirror, permet l'auto-install texte
<code>--network network=default,model=virtio</code>	réseau virtuel libvirt
<code>--osinfo name=debian11</code>	optimisations selon l'OS invité
<code>--graphics none</code>	pas de console graphique
<code>--graphics spice</code>	console SPICE
<code>--console pty,target_type=serial</code>	console série
<code>--extra-args "console=ttyS0,..."</code>	args kernel transmis pour I/O série
<code>--print-xml</code>	génère seulement le XML (pas de création)

Variantes

```
# Install graphique standard
virt-install --name vm2 --memory 2048 --vcpus 2 \
  --disk size=10 --cdrom ~/iso/debian.iso \
  --network network=default --graphics spice

# Importer un disque existant (pas d'install)
virt-install --name vm3 --memory 1024 --vcpus 1 \
  --disk path=/var/lib/libvirt/images/vm3.qcow2 \
  --network network=default --import --osinfo name=debian11
```

4. Stockage (raw, qcow2, LVM)

Formats de disque

Format	Avantages	Inconvénients
raw	rapide, simple, allocation totale	pas de snapshot interne, taille = totale
qcow2	allocation dynamique, snapshots, compression	léger surcoût CPU
LVM (volume logique)	snapshots côté hôte, redimensionnement, perfs	plus complexe à mettre en place

qemu-img

```
qemu-img create -f raw /tmp/vm.img 6G
qemu-img create -f qcow2 /tmp/vm.qcow2 6G
qemu-img info /tmp/vm.qcow2
qemu-img convert -f raw -O qcow2 vm.raw vm.qcow2
qemu-img resize vm.qcow2 +2G # agrandir le fichier (pas la partition)
```

Pools libvirt

```
virsh pool-list --all
virsh pool-dumpxml default
virsh pool-define-as pooldir --type dir --target /var/lib/libvirt/images
virsh pool-create-as poolvm logical --target /dev/vg --source-name vg
virsh pool-start pooldir
virsh pool-autostart pooldir

virsh vol-list default
virsh vol-create-as default vm.img 4G --format raw
virsh vol-upload --pool default vm.img /chemin/local.img
virsh vol-download --pool default vm.img ./backup.img
```

Attacher un disque à chaud

```
sudo qemu-img create -f raw /var/lib/libvirt/images/vol2.img 2G
sudo chown libvirt-qemu:kvm /var/lib/libvirt/images/vol2.img

virsh attach-disk vm1 \
  /var/lib/libvirt/images/vol2.img vdb \
  --driver qemu --subdriver raw \
  --config --live # --config = persistant, --live = à chaud
```

5. Clonage de VMs

Clonage simple (fichier disque)

```
virt-clone --connect qemu:///system \
  --original vm1 \
  --name vm1b \
  --file /var/lib/libvirt/images/vm1b.img
```

virt-clone : 1. copie le disque source → destination, 2. modifie le XML (UUID, nom, MAC réseau, chemins disques), 3. enregistre la nouvelle VM.

Clonage vers un volume LVM (LV existant)

```
# 1. créer le LV cible
sudo lvcreate -L 6G -n kvm-vm3 vg

# 2. copier les données (dd)
sudo dd if=/var/lib/libvirt/images/vm1.raw \
  of=/dev/vg/kvm-vm3 \
  bs=4M status=progress

# 3. virt-clone avec --preserve-data (NE PAS recopier le contenu)
virt-clone --original vm1 --name vm3 \
  --file /dev/vg/kvm-vm3 --preserve-data
```

`--preserve-data` est **essentiel** : sans lui, virt-clone réécrit la cible et écrase le `dd`.

Post-clonage (à faire dans la VM clonée)

- changer `hostname` et `/etc/hosts`
- régénérer les clés SSH hôte : `sudo rm /etc/ssh/ssh_host_*` puis `sudo dpkg-reconfigure openssh-server`
- vérifier `/etc/machine-id` (régénérer si besoin : `sudo systemd-machine-id-setup`)

6. LVM — extension & snapshots

Notions

Élément	Symbole	Rôle
PV	Physical Volume	partition ou disque consacré à LVM
VG	Volume Group	regroupe des PVs
LV	Logical Volume	"partition" logique au sein d'un VG

```
sudo pvs / pvdisplay
sudo vgs / vgdisplay
sudo lvs / lvdisplay
```

Étendre un LV (côté hôte)

```
virsh destroy vm3          # arrêter la VM
sudo lvextend -L +2G /dev/vg/vm3  # +2 Gio
# (puis dans la VM : étendre la partition + filesystem)
```

Étendre une partition + FS dans la VM (sans LVM interne)

```
sudo apt install kpartx
sudo kpartx -av /dev/vg/vm3          # mappe /dev/mapper/vg-vm3p1 etc.
sudo fdisk /dev/vg/vm3              # supprimer/recréer la partition plus grande
sudo kpartx -d /dev/vg/vm3
sudo kpartx -av /dev/vg/vm3
sudo e2fsck -fy /dev/mapper/vg-vm3p1
sudo resize2fs /dev/mapper/vg-vm3p1
sudo kpartx -d /dev/vg/vm3
virsh start vm3
```

Étendre un LV interne (VM avec LVM)

```
# 1. côté hôte
virsh destroy vm4
sudo lvextend -L +2G /dev/vg/vm4
virsh start vm4

# 2. dans la VM
sudo fdisk /dev/vda          # nouvelle partition de type 8e (LVM)
sudo pvcreate /dev/vda3
sudo vgextend vm-vg /dev/vda3
sudo lvextend -l +100%FREE /dev/vm-vg/root
sudo resize2fs /dev/vm-vg/root
```

Snapshots LVM

```
virsh destroy vm3
sudo lvcreate -s /dev/vg/vm3 -L 1G -n vm3-s1 # snapshot COW de 1G

# Revenir à l'état du snapshot (annuler)
virsh destroy vm3
sudo lvconvert --merge /dev/vg/vm3-s1
virsh start vm3

# Conserver les modifications (jeter le snapshot)
sudo lvremove /dev/vg/vm3-s1
```

7. Réseaux virtuels libvirt

Réseau **default** (NAT)

- Pont : **virbr0**
- Plage : **192.168.122.0/24**
- Passerelle : **192.168.122.1**
- DHCP fourni par libvirt

```
<network>
  <name>default</name>
  <forward mode='nat' />
  <bridge name='virbr0' stp='on' delay='0' />
  <ip address='192.168.122.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.122.2' end='192.168.122.254' />
    </dhcp>
  </ip>
</network>
```

Réseau isolé (sans **forward**)

```
<network>
  <name>net-isole</name>
  <bridge name='virbr10' stp='on' delay='0' />
  <ip address='10.10.10.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='10.10.10.128' end='10.10.10.254' />
    </dhcp>
  </ip>
</network>
```

Réservation DHCP fixe (par MAC)

```
<dhcp>
  <range start='192.168.122.128' end='192.168.122.250' />
  <host mac='52:54:00:19:85:3e' name='vm3' ip='192.168.122.254' />
</dhcp>
```

Commandes `virsh net-*`

```
virsh net-list --all
virsh net-define net-isole.xml
virsh net-start net-isole
virsh net-autostart net-isole
virsh net-stop net-isole
virsh net-destroy net-isole
virsh net-edit default # édition XML inline
virsh net-dumpxml default
```

8. Configuration réseau dans la VM

Netplan (Ubuntu, Debian récent)

```
# /etc/netplan/90-default.yaml
network:
  version: 2
  ethernets:
    all-en:
      match:
        name: en*
      dhcp4: true
      routes:
        - to: 10.10.10.0/24
          via: 192.168.122.254
```

```
sudo netplan apply
sudo netplan status
```

`/etc/network/interfaces` (Debian classique)

```
auto lo
iface lo inet loopback

allow-hotplug ens3
iface ens3 inet dhcp
    post-up ip route add 10.10.10.0/24 via 192.168.122.254 || true
    pre-down ip route del 10.10.10.0/24 via 192.168.122.254 || true
```

IP forwarding (passerelle)

```
# Temporaire
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward

# Permanent
sudo sed -i 's/^#\?net.ipv4.ip_forward.*/net.ipv4.ip_forward=1/' /etc/sysctl.conf
sudo sysctl -p
```

Inspection

```
ip a          # adresses
ip -br a      # format compact
ip route      # routes
ip link       # interfaces (UP/DOWN)
ss -tulpn     # ports en écoute
ping -c4 host
traceroute host
```

9. SSH (jump host, tunnel, clés)

Connexion de base

```
ssh user@host
ssh -p 2222 user@host
ssh -i ~/.ssh/maclé user@host
```

Clés SSH (sans mot de passe)

```
ssh-keygen -t ed25519          # ou rsa -b 4096
ssh-copy-id user@host          # installer la clé sur le serveur
# Manuel :
cat ~/.ssh/id_ed25519.pub | ssh user@host \
'mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys'
```

ProxyJump (saut de machine)

```
ssh -J user@bastion user@cible
# équivalent dans ~/.ssh/config :
Host cible
  HostName 10.10.10.199
  User debian
  ProxyJump bastion
```

Tunnel SSH (port forwarding local)

```
# Local 2222 → distant cible:22 via bastion
ssh -N -L 2222:10.10.10.199:22 debian@192.168.122.254
# (-N : pas de shell, -L : local forward)

# Dans un autre terminal :
ssh -p 2222 debian@localhost
```

Tunnel inverse (remote forward)

```
ssh -N -R 9000:localhost:80 user@host
# Le port 9000 du serveur distant tape sur le port 80 local
```

10. Post-installation d'une VM

Hostname

```
sudo hostnamectl set-hostname vm1
# OU manuel :
echo "vm1" | sudo tee /etc/hostname
sudo sed -i "s/127.0.1.1.*/127.0.1.1\tvm1/" /etc/hosts
```

Console série (**virsh console**) — GRUB

```
sudo nano /etc/default/grub
# Modifier :
GRUB_CMDLINE_LINUX_DEFAULT="quiet console=ttyS0,115200n8"
GRUB_TERMINAL=serial
GRUB_SERIAL_COMMAND="serial --speed=115200"

sudo update-grub
sudo reboot
```

Régénérer les clés SSH du serveur

```
sudo rm /etc/ssh/ssh_host_*
sudo dpkg-reconfigure openssh-server
# Vérification
for f in /etc/ssh/ssh_host_*.pub; do ssh-keygen -lf "$f"; done
```

Mises à jour & SSH client

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y openssh-server sudo vim
```

11. Incus (LXD fork)

Installation & init

```
sudo apt install -y incus
sudo incus admin init # configuration interactive
sudo apt install -y incus-ui-canonical # interface web
```

Cycle de vie des instances

```
incus list # toutes les instances
incus launch images:debian/12 c1 # conteneur
incus launch --vm images:debian/12 vm1 # vraie VM
incus stop c1
incus start c1
incus restart c1
incus delete c1 # ne tourne plus → suppression
incus delete --force c1 # forcer

incus exec c1 -- bash # commande à l'intérieur
incus exec c1 -- apt update
incus shell c1 # alias pour `exec -- bash`
incus info c1 # infos détaillées
```

Fichiers, snapshots, copie

```
incus file push    fichier.txt c1/root/  
incus file pull    c1/root/fichier.txt .  
incus snapshot create c1 snap1  
incus snapshot list  c1  
incus snapshot restore c1 snap1  
incus copy    c1 c2 --copy-snapshots  
incus rename c1 c1-old
```

Profils (config réutilisable)

```
incus profile list  
incus profile show default  
incus profile create devp  
incus profile edit    devp  
incus profile assign c1 default,devp  
incus launch -p default -p devp images:debian/12 c2
```

Interface web

```
incus webui          # affiche URL + token à coller dans le navigateur
```

ANNEXES

Cheatsheet 1 page — commandes les plus probables au test

Shell (one-liners)

```
# Compter les fichiers ordinaires sous le dossier courant
find . -type f | wc -l

# Lister les usernames du système triés
cut -d: -f1 /etc/passwd | sort

# Trouver les fichiers > 1 Mio modifiés cette semaine
find . -type f -size +1M -mtime -7

# Lister les UID dans la plage 100..199
cut -d: -f3 /etc/passwd | awk '$1>=100 && $1<=199'

# Donner droits 0644 à tous les .txt sous le dossier
find . -name "*.txt" -exec chmod 0644 {} \;

# Sauvegarder /etc en redirigant l'erreur dans un log
tar -cf etc.tar /etc 2> tar-errors.log

# Compter les lignes uniques d'un fichier
sort fichier | uniq | wc -l

# Lancer 3 commandes : 2nd seulement si 1re OK
make && cp out /usr/local/bin/ && echo "OK"
```

Permissions express

Octal	Litt.	Cas typique
600	rw-----	clés SSH
644	rw-r--r--	fichier texte standard
700	rw-x-----	dossier perso ~/.ssh
755	rwxr-xr-x	dossier ou exécutable public
666	rw-rw-rw-	partage total (rare)
777	rwxrwxrwx	DANGER, à éviter

Virtualisation (one-liners)

```
# Créer une VM Debian en console série
virt-install -n vm1 -r 1024 --vcpus 1 \
  --disk size=6,format=raw,bus=virtio \
  --location ~/iso/debian.iso --network network=default \
  --osinfo name=debian11 --graphics none \
  --console pty,target_type=serial \
  --extra-args "console=ttyS0,115200n8"

# Cloner une VM (fichier)
virt-clone --original vm1 --name vm2 --file /var/lib/libvirt/images/vm2.img

# Snapshot LVM + revert
sudo lvcreate -s /dev/vg/vm3 -L 1G -n vm3-snap
sudo lvconvert --merge /dev/vg/vm3-snap

# Réserve DHCP (à coller dans virsh net-edit default)
<host mac='52:54:00:19:85:3e' name='vm3' ip='192.168.122.254' />

# Tunnel SSH via passerelle
ssh -J debian@192.168.122.254 debian@10.10.10.199
```

Pièges fréquents au test

1. **find sans guillemets** : `find . -name *.h` casse si un `*.h` existe localement (le shell expand). → Toujours `'*.h'` ou `"*.h"`.
2. **usermod -G groupe** sans `-a` : **remplace** tous les groupes secondaires. Toujours `usermod -aG`.
3. **chmod g+w sur dossier** sans `+x` : write inutile, le `x` est requis pour entrer.
4. **virt-clone vers LVM** sans `--preserve-data` : virt-clone écrase le `dd` que vous venez de faire.
5. **Console série KVM** sans `console=ttyS0` dans GRUB : `virsh console` reste muet.
6. **Clone de VM sans régénérer clés SSH/ machine-id** : deux VMs avec la même identité, conflit DHCP, warnings SSH.
7. **shutdown vs destroy** : `shutdown` est ACPI (propre), `destroy` est l'équivalent d'un débranchage électrique.
8. **umask est soustractif**, pas additif : `umask 0022` → fichier 644, dossier 755.
9. **Variables non exportées** : un script enfant ne les voit pas. Utiliser `export` ou sourcer le script (`. script.sh`).
10. **> écrase**, **>> ajoute**. Toujours vérifier avant un `>` sur un fichier existant.
11. **Permissions octales sur 4 chiffres** : `chmod 0644` (le 0 = pas de bit spécial). Bits spéciaux : `4xxx` setuid, `2xxx` setgid, `1xxx` sticky.
12. **Réseau isolé** : pas de `<forward>` dans le XML. Les VMs ne sortent pas (sauf via une VM passerelle avec IP forwarding activé).

Bonne révision et bon test !